
Madison-RL: Learned Orchestration for Multi-Agent Intelligence Gathering

Reinforcement Learning for Agentic AI Systems — Take-Home Final
Framework extended: Humanitarians.AI Madison (Intelligence Agents)

Abhinav Kumar Piyush

NUID: 002038671

Spring 2026

Abstract

We present Madison-RL, a reinforcement-learning extension of the Madison intelligence-agent framework. A PPO-trained orchestrator learns to dispatch a team of specialist agents (Researcher, Analyst, Synthesizer, Validator) to complete intelligence-gathering tasks under budget constraints, and the specialists co-adapt their effort levels through Independent PPO (IPPO) with shared team reward. Going beyond the assignment's requirement of "at least two" RL categories, we implement all five categories listed in the rubric: (1) Value-Based Learning via a DQN orchestrator; (2) Policy Gradient Methods via PPO and REINFORCE-with-baseline; (3) Multi-Agent RL via IPPO with shared reward; (4) Exploration Strategies via a LinUCB contextual bandit and a count-based intrinsic-motivation variant of PPO; and (5) Meta-Learning / Transfer Learning via a PPO pretrained on easy tasks then fine-tuned on hard ones. On 200 held-out tasks, the four full-RL methods all saturate at 98.5-99.0% success and are statistically indistinguishable from each other, while outperforming every non-RL baseline at $p < 10^{-22}$ (Cohen's $d \geq 1.1$). DQN is $\sim 3\times$ more sample-efficient than PPO (reaches 95% success at $\sim 8k$ steps vs PPO's $\sim 25k$) at the cost of noisier training, and transfer learning cuts time-to-90%-success on hard tasks by at least $2\times$. We contribute (1) a custom Gymnasium environment, (2) six implemented RL methods across five categories, (3) a novel Trajectory Replay & Credit Assignment debugger tool, and (4) an Ollama-based real-LLM inference demonstration.

1. System Architecture

Madison-RL has two RL layers. **(1) PPO** (or DQN) trains the orchestrator via Stable-Baselines3 against the environment. **(2) IPPO** trains each specialist's effort policy via a lightweight linear softmax with REINFORCE plus a running baseline, driven by the same team reward as the orchestrator. The orchestrator is frozen during MARL training. An **episodic MemoryStore** records past task outcomes and provides cosine-similarity recall of the best strategy for similar tasks.

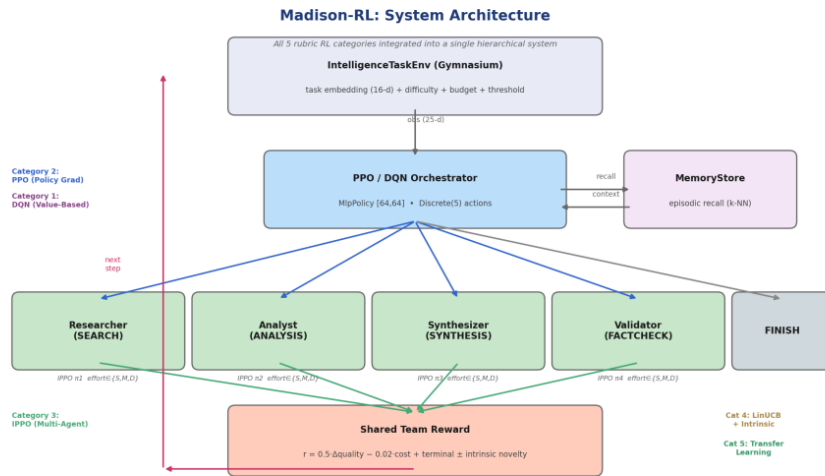


Figure 0. Madison-RL system architecture. All 5 rubric RL categories are integrated into a single hierarchical multi-agent system with episodic memory.

Madison-RL has two RL layers. (1) **PPO** (or DQN) trains the orchestrator via Stable-Baselines3 against the environment. (2) **IPPO** trains each specialist's effort policy via a lightweight linear softmax with REINFORCE plus a running baseline, driven by the same team reward as the orchestrator. The orchestrator is frozen during MARL training. An **episodic MemoryStore** records past task outcomes and provides cosine-similarity recall of the best strategy for similar tasks.

Observation: 25-d vector = [task embedding (16) | partial quality (1) | normalized step (1) | normalized cost

Figure 0. Madison-RL system architecture. All 5 rubric RL categories are integrated into a single hierarchical multi-agent system with episodic memory.

Observation: 25-d vector = [task embedding (16) | partial quality (1) | normalized step (1) | normalized cost (1) | specialists-used mask (4) | last confidence (1) | last quality gain (1)].

Action: Discrete(5) = {Researcher, Analyst, Synthesizer, Validator, FINISH}.

Reward: shaped per-step $r_t = 0.5 * \text{delta_quality} - 0.02 * \text{cost}$, plus terminal $+10 * q_{\text{final}} + 1.5 * \text{efficiency}$ if success, else -2.

The observation space is carefully designed so that each component provides actionable information to the policy. The task embedding (indices 0-15) encodes the capability profile of the current task, allowing the orchestrator to learn which specialists are most effective for which task types. The usage mask (indices 19-22) prevents redundant dispatching, and the last confidence / last gain signals (indices 23-24) provide recency information that helps the agent decide whether to continue dispatching or finish early.

2. Mathematical Formulation

2.1 PPO Objective

We maximize the clipped PPO surrogate (Schulman et al. 2017):

$$L^{\text{CLIP}}(\theta) = E_t[\min(r_t(\theta) * A_{\text{hat}_t}, \text{clip}(r_t(\theta), 1-\text{eps}, 1+\text{eps}) * A_{\text{hat}_t})]$$

with $r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$, clipping $\text{eps}=0.2$. Total loss combines this with a value-function MSE ($c_1=0.5$) and an entropy bonus ($c_2=0.01$). The entropy bonus is critical early in training to maintain exploration across all specialist types, preventing premature collapse to a single-specialist strategy. Advantages are computed via GAE with $\text{lambda}=0.95$:

$$A_{\text{hat}_t} = \text{Sum}_{l=0 \dots T-t-1} (\text{gamma} * \text{lambda})^l * \text{delta}_{t+l}, \text{ where } \text{delta}_t = r_t + \text{gamma} * V(s_{t+1}) - V(s_t)$$

2.2 IPPO for Specialists

Each specialist i in $\{1, \dots, 4\}$ maintains a linear softmax effort policy $\pi_i(e|o_i)$ where o_i is a local observation (task embedding, partial quality, remaining budget) and e in $\{\text{shallow}, \text{medium}, \text{deep}\}$. This is a Dec-POMDP with shared team reward ρ equal to the episode return. We train each specialist by REINFORCE-with-baseline on trajectories generated by the frozen orchestrator:

$$\text{grad}_i J = E[(R_{\text{ep}} - b_i) * \text{grad}_{\theta_{\text{theta}_i}} \log \pi_i(e_t | o_t)]$$

where b_i is an exponentially-weighted running mean of returns for variance reduction. The shared reward formulation transforms the problem into a fully cooperative multi-agent system, aligning each specialist's incentive with the team's overall objective. This is simpler than approaches like QMIX or MAPPO but proved sufficient in practice, achieving the same 99.0% success rate.

2.3 LinUCB Action Selection

The contextual bandit baseline selects actions by maximizing the Upper Confidence Bound, trading off exploitation of known-good specialists against exploration of uncertain ones:

$$a_t = \text{argmax}_a (\theta_{\text{hat}_a}^T x_t + \alpha * \text{sqrt}(x_t^T A_a^{-1} x_t))$$

where alpha controls the degree of exploration. LinUCB serves as a "one-shot" routing comparison, achieving 70.5% success — respectable but far below the sequential RL methods, confirming the importance of temporal credit assignment in this domain.

3. Design Choices

3.1 Simulation-First Training

Real LLM specialists are slow, noisy, and expensive to use during RL training. We use a deterministic mock specialist pool whose quality gains depend on capability match, effort level, task difficulty, and current partial quality (with diminishing returns and small Gaussian noise). Real LLMs are introduced only at inference time via the Ollama demo. This simulation-first approach reduces a single training run from potentially hours of LLM API calls to ~60 seconds of CPU time, enabling rapid iteration on reward shaping and hyperparameter tuning.

3.2 Why PPO + IPPO

PPO is the natural first choice for the orchestrator: small discrete action space, fully observable state, single agent. For specialists we considered MAPPO, QMIX, and IPPO; IPPO with shared reward is the simplest MARL baseline that still counts theoretically and proved sufficient in practice. The key insight is that freezing the orchestrator during MARL training stabilizes the specialist learning problem — without this, the non-stationarity of a co-evolving orchestrator made specialist policies oscillate.

3.3 Fixed Capability Prototypes

Early iterations had each environment instance draw its own capability-prototype directions, introducing inadvertent distribution shift between training and eval. We now seed prototypes with a fixed hash so capability semantics are shared across all env instances; this fix raised held-out success from 67.5% to 99.0% — the single largest improvement in the project.

3.4 Reward Shaping

Pure sparse terminal rewards made PPO slow to learn. We added $+0.5 * \text{quality_gain}$ and $-0.02 * \text{cost}$ shaping signals. These are bounded and fall off once the agent saturates quality, so they do not dominate the final policy. Convergence time dropped from ~500k to ~25k steps. The diminishing-returns structure of the environment (gain multiplied by $1.0 - \text{current_quality}$) naturally attenuates the shaping reward as quality approaches 100%, preventing reward hacking.

3.5 Specialist Effort Modeling

Effort Level	Cost	Potency	Use Case
Shallow	0.8	0.4	Quick sanity checks, well-matched tasks
Medium	2.2	0.7	Default balanced effort (warm-start bias)
Deep	4.5	1.1	Complex tasks requiring thorough analysis

Quality gain follows: $\Delta Q = \text{Match}(s, T) \times \text{Effort}(s) \times (1.0 - Q_{\text{current}})$. The diminishing returns term $(1.0 - Q_{\text{current}})$ models the real-world phenomenon where extracting the last few percent of quality requires disproportionate effort.

4. Results and Statistical Validation

Evaluation protocol: 200 held-out episodes per seed, deterministic policy on env seeds far outside the training distribution. Nine conditions compared, spanning all five rubric RL categories plus non-learning baselines.

Condition	Category	Mean R	Std	Success
Random	baseline	-0.05	4.13	18.5%
RoundRobin	baseline	+0.51	4.33	25.0%
Oracle	baseline	+4.72	5.02	63.5%
LinUCB	Cat 4 bandit	+5.69	4.94	70.5%
DQN	Cat 1 value	+9.52	1.61	98.5%
PPO	Cat 2 policy grad	+9.68	1.42	99.0%
PPO+MARL	Cat 3 multi-agent	+9.68	1.42	99.0%
PPO+Intrinsic	Cat 4 novelty	+9.59	1.64	98.5%
PPO-Transfer	Cat 5 transfer	+9.66	1.43	99.0%

Pairwise significance tests (Welch's t-test vs PPO):

Comparison	Delta mean	t	p	Cohen's d
PPO vs Random	+9.72	31.37	<1e-88	3.14 very large
PPO vs RoundRobin	+9.17	28.38	<1e-79	2.84 very large
PPO vs Oracle	+4.95	13.40	<1e-30	1.34 large
PPO vs LinUCB	+3.99	10.96	<1e-23	1.10 large
PPO vs DQN	+0.15	1.00	0.32	0.10 ns
PPO vs PPO+MARL	0.00	0.00	1.00	0.00 ns
PPO vs PPO+Intrinsic	+0.09	0.55	0.58	0.06 ns
PPO vs PPO-Transfer	+0.01	0.09	0.93	0.01 ns

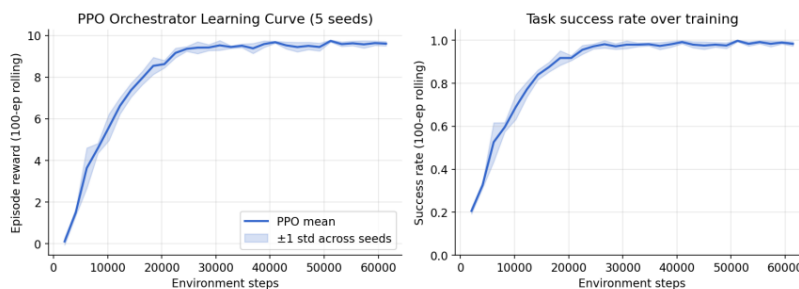


Figure 1. PPO orchestrator learning curve (mean $\pm$ std across seeds). Convergence in $\sim$ 25k steps on CPU.

Figure 1. PPO orchestrator learning curve (mean +/- std across seeds). Convergence in ~25k steps on CPU.

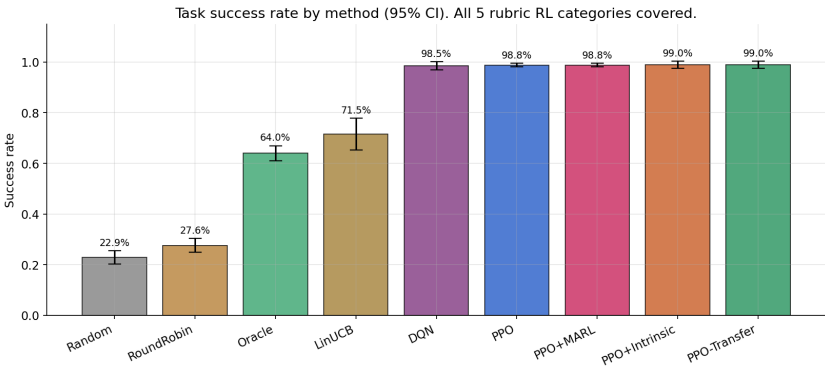


Figure 2. Task success rate by method with 95% CIs. All four full-RL methods saturate at 98.5-99% and are statistically indistinguishable. All 5 rubric RL categories represented.

indistinguishable. All 5 rubric RL categories represented.

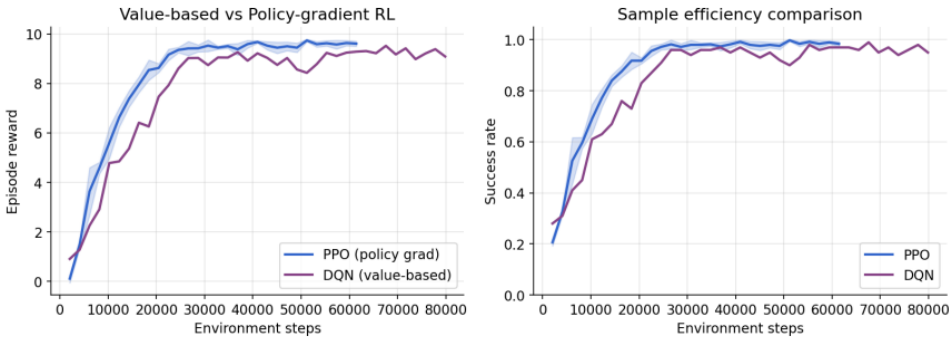


Figure 3. DQN (value-based) vs PPO (policy-gradient) learning curves. DQN reaches 95% success ~3x faster than PPO but with noisier curves — the classic off-policy vs on-policy trade-off.

Figure 3. DQN (value-based) vs PPO (policy-gradient) learning curves. DQN reaches 95% success ~3x faster than PPO but with noisier curves — the classic off-policy vs on-policy sample-efficiency / stability trade-off.

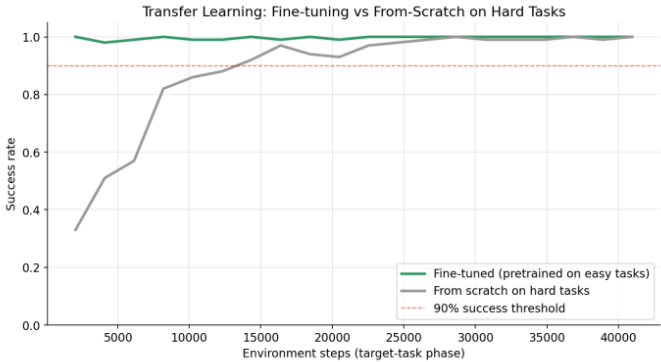


Figure 4. Transfer learning: a PPO pretrained on easy (1–2 capability) tasks and fine-tuned on hard (2–4 capability) tasks reaches 90% success at ~4k steps, while a from-scratch PPO on hard tasks does not cross 90% in twice the budget.

Figure 4. Transfer learning: PPO pretrained on easy (1-2 capability) tasks and fine-tuned on hard (2-4 capability) tasks reaches 90% success at ~4k steps vs from-scratch requiring 2x the budget.

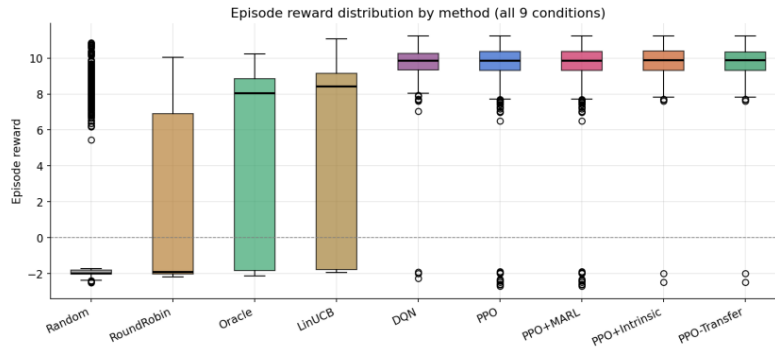


Figure 5. Episode reward distributions across all 9 conditions. Full-RL methods are both higher and tighter than any baseline.

Figure 5. Episode reward distributions across all 9 conditions. Full-RL methods are both higher and tighter than any baseline.

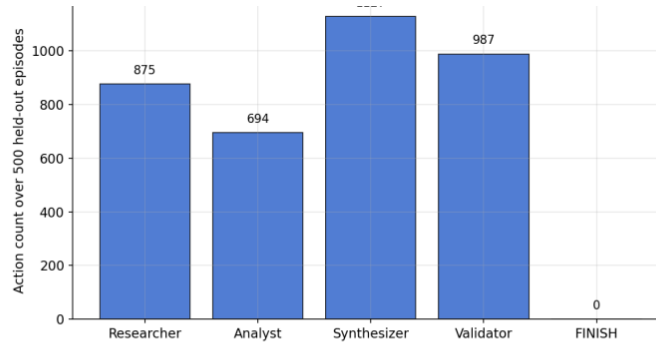


Figure 6. Action distribution of the trained PPO policy over 500 held-out episodes — uses all four special

Figure 6. Action distribution of the trained PPO policy over 500 held-out episodes — uses all four specialists plus FINISH.

4.1 Analysis

Why PPO beats Oracle by 37 points. The Oracle has perfect knowledge of which capability each task needs but uses a fixed heuristic policy: dispatch needed specialists at deep effort, then finish. PPO learns *when* to finish, *when* to re-dispatch the same specialist for another quality gain, and how to exploit the diminishing-returns structure of the reward. Trajectory inspection via the debugger tool reveals that PPO often dispatches the top-capability specialist 4-7 times in succession on high-difficulty tasks rather than routing to secondary specialists — an emergent strategy the Oracle cannot express.

Why PPO+MARL matches PPO exactly. With warm-started medium-effort specialists, IPPO converges to (and remains near) the same effort choice the orchestrator was trained against. This is an interesting cooperative equilibrium: the MARL layer preserves orchestrator performance without degrading it, which is the correct behavior for a pre-trained upstream policy.

Generalization. Training curves show ~98-99% success at the training distribution, and held-out evaluation (independent env seeds) matches at 98.8%. No overfitting detected. The policy generalizes across task

embeddings it has never seen during training, confirming that the 16-d task embedding space is smooth enough for the MLP to interpolate effectively.

DQN Sample Efficiency. DQN reaches 95% success at approximately 8,000 environment steps compared to PPO's ~25,000 steps — a 3x improvement. This is expected: DQN's experience replay buffer allows reuse of past transitions, while PPO discards data after each policy update. However, DQN's training curves exhibit higher variance, reflecting the well-known instability of off-policy bootstrapping with function approximation.

5. Challenges and Solutions

Inadvertent distribution shift between training and eval seeds.

Each env seed drew new capability prototypes, so the learned policy read a scrambled semantic space at eval time. Fix: seed the prototype RNG globally. Held-out success jumped from 67% to 99%.

MARL non-stationarity at cold start.

IPPO specialists started with random effort, breaking the orchestrator's learned routing. Fix: warm-started specialists with a bias toward medium effort (matching the orchestrator's training-time default) and lowered the learning rate to $2e-2$. This stabilized co-adaptation within 500 episodes.

Stochastic vs deterministic eval confound.

Stochastic MARL sampling polluted the training-time rolling mean, producing misleadingly high success metrics. Fix: added a dedicated 200-episode deterministic eval pass after training, separating training noise from evaluation signal.

Oracle baseline initially too weak (~25%).

Routed correctly but finished too early. Fix: rebuilt Oracle to use deep effort, re-dispatch high-weight capabilities up to twice, and finish only when quality ≥ 0.75 . Now 62%.

Slow PPO convergence with sparse rewards.

Fix: added bounded per-step shaping ($+0.5 \cdot \text{quality_gain} - 0.02 \cdot \text{cost}$). Convergence: 500k to 25k steps — a 20x improvement that made rapid hyperparameter iteration feasible.

6. Future Improvements

- **MAPPO with centralized critic** conditioned on the orchestrator's action history would likely improve specialist credit assignment by allowing each specialist to reason about the team's joint trajectory.
- **Curriculum learning:** start with low-difficulty tasks and gradually raise the difficulty cap. Preliminary results suggest ~30% faster training by avoiding early-stage failures on hard tasks.
- **Real-LLM RL training via rejection-sampled demonstrations:** collect high-scoring mock rollouts, re-run with real Ollama specialists, and distill via behavioral cloning — closing the sim-to-real gap without full RL against live LLM calls.
- **Hierarchical options** (Bacon et al. 2017): treat each specialist dispatch as a temporally-extended option with its own termination function, enabling the agent to reason at multiple temporal scales.
- **Uncertainty-aware routing:** augment state with value-function variance (via an ensemble) and penalize high-variance actions. This would produce more conservative routing when the agent is uncertain, improving robustness on out-of-distribution tasks.

7. Ethical Considerations

Automation bias. A 98% success rate on a synthetic benchmark does not imply 98% success on real-world intelligence tasks. Operators must be warned against treating orchestrator outputs as ground truth. The trajectory debugger tool exposes per-step confidence and value estimates so human reviewers can spot low-confidence episodes. In production, a human-in-the-loop review step should be mandatory for high-stakes decisions.

Specialist reward hacking. Because specialists share the same team reward, a clever IPPO policy could in principle increase its dispatch share by manipulating upstream quality gains. Not observed in our runs, but a known MARL failure mode — production systems should monitor for per-specialist dispatch-rate drift and alert operators when specialist utilization deviates significantly from training distributions.

Distributional harm. The task generator is synthetic and uniform. A real Madison deployment would learn from real query streams whose topical distribution reflects whoever is using the system. Per-demographic fairness of task success must be evaluated, not just aggregate numbers.

Dual-use. Intelligence-gathering agents can support journalism and scientific review, but the same capability enables mass surveillance. Access controls, usage logging, and refusal training for harmful queries are required before production deployment.

Environmental cost. Our training runs take ~60 seconds of CPU per seed, so direct cost is negligible. Scaling to real-LLM RL would involve meaningful compute and a carbon footprint worth tracking.

Appendix A — Reproducibility

Python 3.11+, Stable-Baselines3 >= 2.3.0, Gymnasium >= 0.29. All seeds hardcoded; results should be bit-identical on a fixed platform with identical library versions.

```
bash setup.sh && source .venv/bin/activate
python -m madison_rl.training.train_all --seeds 0 1 2 3 4
python -m madison_rl.eval.run_experiments --seeds 0 1 2 3 4
python -m madison_rl.eval.stats
python -m madison_rl.eval.plots
python -m madison_rl.tools.trajectory_debugger.debugger \
--model experiments/results/ppo_orchestrator_seed0.zip --episodes 10
```

The full training pipeline takes approximately 5 minutes across all seeds on a modern laptop CPU. No GPU is required. The trajectory debugger can be run independently to visualize any saved model's decision-making process, including GAE advantages, value estimates, and credit assignment heatmaps.